

Input Format for Custom Testing

▼ Sample Case 0

Sample Input 0

STDIN

Function

julasamantantjulia

→ text = 'julasamantantjulia'

ant

→ searchPattern = 'ant'

Sample Output 0

8

Explanation 0

text	j	u	l	i	a	s	a	m	a	n	t	h	a	n	t	j	u	l	i	a
searchPattern										a	n	t								
Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

▼ Sample Case 1

Sample Input 1

STDIN

Function

julasamantasantanjulia

→ text = 'julasamantasantanjulia'

has

→ searchPattern = 'has'

Sample Output 1

11

Explanation 1

text	j	u	l	i	a	s	a	m	a	n	t	h	a	s	a	m	a	n	t	h	a
searchPattern												h	a	s							
Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

► Sample Case 2

Language C++14

```
1 > #include <bits/stdc++.h>
2
3
4
5 /*
6  * Complete the 'firstOccurrence' function below.
7  *
8  * The function is expected to return an INTEGER.
9  * The function accepts following parameters:
10  * 1. STRING text
11  * 2. STRING searchPattern
12  */
13
14
15
16 int firstOccurrence(string text, string searchPattern) {
17
18 }
19
20 > int main()...
```

1 hr 10 min



S1



S2



S3

3

S4

4

Given a string `text` and a pattern `searchPattern`, find the zero-based index of the first occurrence of `searchPattern` in `text`.

- The string `text` contains only lowercase letters (a-z).
- The pattern `searchPattern` consists of lowercase letters and may include a single wildcard character `*`, which matches any one character.

Example

`text = "xabcdey"`
`searchPattern = "ab*de"`

The first match is at index 1.

text	x	a	b	c	d	e	y
searchPattern		a	b	*	d	e	
Index	0	1	2	3	4	5	6

Function Description

Complete the function `firstOccurrence` in the editor, with the following parameters:

`string text`: the text to search

`string searchPattern`: the pattern to search for

Return

`int`: the zero-based index of the first occurrence of string `searchPattern` in `text`. If `searchPattern` is not in `text`, return `-1` instead.

Constraints

- $1 \leq \text{length of text} \leq 5 \times 10^5$
- $1 \leq \text{length of searchPattern} \leq 1000$

► Input Format for Custom Testing

▼ Sample Case 0

Sample Input 0

STDIN

Function

juliasamanthantjulia → `text = 'juliasamanthantjulia'`

Layout

Save & P

Language

C++14

```
1 > #include <bits/stdc++.h>
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Complete the 'firstOccurrence' function below.

The function is expected to return an INTEGER.

The function accepts following parameters:

- 1. STRING `text`
- 2. STRING `searchPattern`

```
int firstOccurrence(string text, string searchPattern) {
    //
}

int main() ...
```

Test Results

1 hr 24 min



Layout 80



Save & Exit



Input Format for Custom Testing

Sample Case 0

Sample Input 0

STDIN	FUNCTION
3	memoryBlocks[] size n = 3
0	memoryBlocks = [0, 2, 2]
2	
2	

Sample Output 0

0
1

Explanation

If no operation is performed, then the Valid Size is 1.
Selected index $x = 0$ and increase the size of `memoryBlocks[x]` by 1, then `memoryBlocks` becomes [1, 2, 2].
Now the possible Valid Size is 0.
Hence, the answer is [0, 1] which is sorted in ascending order.

Sample Case 1

Sample Input 1

STDIN	FUNCTION
3	memoryBlocks[] size n = 3
2	memoryBlocks = [2, 2, 2]
2	
2	

Sample Output 1

0

Explanation

With no operation on `memoryBlocks`, the possible Valid Size is 0, which is the MEX of `memoryBlocks`.
There is no set of operations through which the MEX is 1 or higher, hence the only possible Valid Size is 0.

Language C++14

```

16 vector<int> findValidSizes(vector<int> memoryBlock) {
17     int n = memoryBlock.size();
18     vector<int> cnt(n+1,0);
19     for(int x; x<memoryBlock.size(); x++){
20         if(x<n) cnt[x]++;
21     }
22
23     int mex=0;
24     while(mex<n and cnt[mex]>0){
25         mex++;
26     }
27     set<int> valid_sz;
28     valid_sz.insert(mex);
29     for(int v{}; v<mex; v++){
30         if(cnt[v] == 1){
31             valid_sz.insert(v);
32         }
33     }

```

Ln 36, Col 30

Autocom

Test Results

All Cases Custom

3 / 15 test cases passed

✗ Test Case 3

✗ Test Case 5

✗ Test Case 6

✗ Test Case 7

✗ Test Case 8

✗ Test Case 9

Compiler Message

Success

Input (stdin)

1 3
2 0
3 2
4 2
5

Input Format for Custom Testing

Sample Case 0

Sample Input 0

STDIN	FUNCTION
3	→ memoryBlocks[] size n = 3
0	→ memoryBlocks = [0, 2, 2]
2	
2	

Sample Output 0

0
1

Explanation

If no operation is performed, then the Valid Size is 1.
Selected index $x = 0$ and increase the size of `memoryBlocks[x]` by 1, then `memoryBlocks` becomes [1, 2, 2].
Now the possible Valid Size is 0.
Hence, the answer is [0, 1] which is sorted in ascending order.

Sample Case 1

Sample Input 1

STDIN	FUNCTION
3	→ memoryBlocks[] size n = 3
2	→ memoryBlocks = [2, 2, 2]
2	
2	

Sample Output 1

0

Explanation

With no operation on `memoryBlocks`, the possible Valid Size is 0, which is the MEX of `memoryBlocks`.
There is no set of operations through which the MEX is 1 or higher, hence the only possible Valid Size is 0.

Language C++14

```
1 > #include <bits/stdc++.h> ...
9
10
11 /* Complete the 'findValidSizes' function below
12 * The function is expected to return an INTEGER_ARRAY
13 * The function accepts INTEGER_ARRAY memoryBlocks as parameter
14 */
15
16 vector<int> findValidSizes(vector<int> memoryBlocks) {
17
18
19 }
20
21 > int main() { ...
```

^ Test Results

Question 3

There are n memory blocks, and the size of the i^{th} block is given by the array element `memoryBlocks[i]`, where $0 \leq i < n$.

The following operation can be performed on `memoryBlocks` one time:

- An index x can be selected. The size of `memoryBlocks[x]` can be increased by 1 unit, but only if `memoryBlocks[x]` is less than $n-1$. The smallest integer k not present in `memoryBlocks` after any number of operations is called a *Valid Size* or the *MEX* (minimum excluded value).

The task is to return an array of all possible *Valid Sizes* that can be achieved using `memoryBlocks`, sorted in ascending order.

Note:

- The *MEX* of `memoryBlocks` is the smallest non-negative integer not present in the array.

Example

Given $n = 3$,
`memoryBlock = [0, 3, 4]`

If no operation on `memoryBlocks` is performed, then the *Valid Size* is 1.

If $x = 0$, increase the size of `memoryBlocks[x]` by 1, then the *Valid Size* is 0.

There are only two possible *Valid Sizes*. Hence, the answer is `[0, 1]`, when sorted in ascending order.

Function Description

Complete the function `findValidSizes` in the editor below.

Function Parameters

`int memoryBlocks[n]`: the memory block sizes

Returns

`int[]`: all possible *Valid Sizes* that can be allocated using `memoryBlocks`, sorted in ascending order

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq \text{memoryBlocks}[i] < n$

Language C++14

```
1 > #include <iostream>
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21 > int main()...
```

^ Test Results

Function Description

Complete the `isValid` function with the following parameters:
`int a[n]`: An array of integers, the values in the order encountered in the traversal of the tree

Returns

string: either the string YES if the path represents a valid BST, or NO

Constraints

- $1 \leq q \leq 10$
- $1 \leq n, a[i] \leq 100$

Input Format

Sample Case 0

Sample Input 0

STDIN	Function
5	→ number of queries $q = 5$
3	→ $a[]$ size $n = 3$ (query 0)
1 3 2	→ $a = [1, 3, 2]$
3	→ $a[]$ size $n = 3$ (query 1)
2 1 3	→ $a = [2, 1, 3]$
6	→ $a[]$ size $n = 6$ (query 2)
3 2 1 5 4 6	→ $a = [3, 2, 1, 5, 4, 6]$
4	→ $a[]$ size $n = 4$ (query 3)
1 3 4 2	→ $a = [1, 3, 4, 2]$
5	→ $a[]$ size $n = 5$ (query 4)
3 4 5 1 2	→ $a = [3, 4, 5, 1, 2]$

Sample Output 0

YES
YES
YES
NO
NO

Explanation 0

Language C++14

```
1 > #include <bits/stdc++.h>
10
11 /*
12  * Complete the 'isValid' function below.
13  * The function is expected to return a STRING.
14  * The function accepts INTEGER_ARRAY a as parameter.
15  */
16
17 string isValid(vector<int> a) {
18
19
20 }
21
22 > int main() ...
```

^ Test Results

1 hr 43 min



S1

S2

2

S3

3

S4

4

Given a pre-order traversal sequence of a binary tree, determine if it represents a binary search tree (BST). A binary tree consists of nodes, each having up to two child nodes and a value. A BST must satisfy two conditions:

1. The left subtree of a node contains only values less than the node's value.
2. The right subtree of a node contains only values greater than the node's value.

Pre-order traversal visits the root node first, then recursively visits the left subtree, followed by the right subtree. Here's the pseudocode for pre-order traversal:

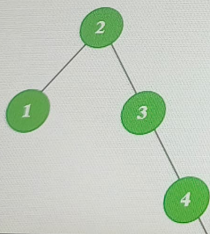
- If the root is null, return an empty list.
- For a non-null root node:
 - Create a pre-order traversal list for the left subtree.
 - Create a pre-order traversal list for the right subtree.
 - Return the concatenated list: node + left subtree + right subtree.

Given a pre-order traversal sequence, check if it can represent a valid BST. Return "YES" if it can, or "NO" if it cannot.

Example

nodes = [2, 1, 3, 4, 5]

- The root node will always be the first node in the array. In this case, the root is at node 2
- The next value, 1 is less than 2. Place the node 1 at the left of node 2.
- The next value, 3 is greater than 2. Place the node 3 at the right of node 2.
- The next value, 4 is greater than 3. Place the node 4 at the right of node 3.
- The next value, 5 is greater than 4. Place the node 5 at the right of node 4.
- This graph meets the definition of a BST.



Language C++14

```

1 > #include <bits/stdc++.h>
10
11
12 /* Complete the 'isValid' function below.
13 * The function is expected to return a STRING.
14 * The function accepts INTEGER_ARRAY a as parameter.
15 */
16
17 string isValid(vector<int> a) {
18
19
20 }
21
22 > int main() {

```

Test Results

▼ Sample Case 0

Sample Input For Custom Testing

STDIN	FUNCTION
2	n = 2
3	m = 3
0 0 0	maze = [[0,0,0],[1,0,0]]
1 0 0	
5	k = 5

Sample Output

2

Explanation

The maze can be represented as:

0	0	0
1	0	0

The following sequence of moves can be performed: (0, 0) → (0, 2) → (1, 2).

▼ Sample Case 1

Sample Input For Custom Testing

STDIN	FUNCTION
2	n = 2
3	m = 3
0 1 0	maze = [[0,1,0],[1,0,0]]
1 0 0	
5	k = 5

Sample Output

-1

Explanation

The maze can be represented as:

0	1	0
1	0	0

HackerMan is blocked and cannot move from the cell (0, 0).

Language C++20

```

20 int getMinimumMoves(vector<vector<int>>& maze, int k) {
21     int dr[] = {0,0,1,-1};
22     int dc[] = {1,-1,0,0};
23
24     while(!q.empty()){
25         pair<int,int> cur = q.front();
26         q.pop();
27         int r = cur.first;
28         int c = cur.second;
29         if(r==n-1 && c==m-1) return dist[r][c];
30         for(int i=0; i<4; i++){
31             for(int step=1; step<=k; step++){
32                 int nr = r+dr[i]*step;
33                 int nc = c+dc[i]*step;
34                 if(nr<0 || nr>=n || nc<0 || nc>=m || maze[nr][nc]==1) break;
35             }
36         }
37     }
38 }

```

Ln 39, Col 50

Autocomplete

Space

▼ Test Results

All Cases Custom

5 / 15 test cases passed

✗ Test Case 1

✗ Test Case 3

✗ Test Case 4

✗ Test Case 6

✗ Test Case 7

✗ Test Case 8

Compiler Message

Wrong Answer

Input (stdin)

```

1 2
2 3
3 0 0 0
4 1 0 0

```




Question 1

In a grid-based maze, each cell is either empty (0) or contains an obstacle (1).

HackerMan must navigate from cell (0, 0) to cell (n-1, m-1) with a jump parameter k , allowing these moves:

- Right: $(i, j) \rightarrow (i, j+x)$ where $1 \leq x \leq k$
- Down: $(i, j) \rightarrow (i+x, j)$ where $1 \leq x \leq k$
- Left: $(i, j) \rightarrow (i, j-x)$ where $1 \leq x \leq k$
- Up: $(i, j) \rightarrow (i-x, j)$ where $1 \leq x \leq k$

For any move, all cells in the path must be obstacle-free and within maze boundaries.

Determine the minimum number of moves required to reach the destination, or return -1 if it is impossible.

Example

Consider $n = 2$, $m = 2$, jump parameter $k = 2$, and $\text{maze} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$.

The maze looks like this.

$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$$

The following sequence of moves can be performed: $(0, 0) \rightarrow (0, 1) \rightarrow (1, 1)$. Hence, HackerMan can reach the end in 2 moves, which is the minimum possible. The answer is 2.

Function Description

Complete the function `getMinimumMoves` in the editor with the following parameters:

`int maze[n][m]`: the maze in HackerPlay where HackerMan is standing
`int k`: the maximum distance HackerMan can traverse in one move

Returns

`int`: the minimum number of moves in which HackerMan can reach the destination cell $(n-1, m-1)$

Constraints

- $1 \leq n \leq 100$
- $1 \leq m \leq 100$
- $1 \leq k \leq 100$



Language

C++20

```

1 > #include <bits/stdc++.h>
10
11
12 /*
13  * Complete the 'getMinimumMoves' function below.
14  * The function is expected to return an INTEGER.
15  * The function accepts following parameters:
16  * 1. 2D_INTEGER_ARRAY maze
17  * 2. INTEGER k
18  */
19
20 Drag to resize getMinimumMoves(vector<vector<int>> maze, int k) {
21
22     }
23
24 > int main() ...

```